SWS3009B – Tele-Robotic

Everything Together Now

External Battery Power

- The motor shield can take external power to boost performance:
  - e.g. batteries etc, up to **12v**!
- External power source is a **fire hazard!**
  - Be careful and follow the instructions!
  - Take note of the **fire safety**

Fire Safety

- When not in use:
  - **Remove the batteries / Power Off**
  - Especially when you are not around

- **At all time:**
  - Be careful of the wiring of the batteries case
  - **Do not place any flammable materials near the batteries case**

Move Like the Real Thing

- Write a program to use serial inputs for controlling the motors as follows:
  - 'W' = move forward at full speed
    - 'w' = move forward at medium/slow speed
  - 'S' = move backward at full speed
    - 's' = move backward at medium/slow speed
  - 'a' = turn left
    - you can make it as close to 90 degree as possible
  - 'd' = turn right
    - similar idea, can make it more accurate
  - 'x' = stop

Be Careful:

Hold the vehicle at all times!

Obstacle Avoidance Vehicle

- Let's utilize another sensor input to guide the vehicle (ultrasonic)

- Write an Arduino sketch to:
  - Move the vehicle forward (SLOWLY!)
  - When obstacle is detected in front, turn LEFT until you find a clear path
  - You can use any reasonable stopping condition:
    - e.g. after moving for 5 seconds, made 10 turns, etc

We have a basic car at this point

- Program **C1** and **C2** is a good starting point for subsequent working:
  - Make sure you spend time to improve them
- You can consider combining them
- Break your Arduino program into functions, don't write everything in **`loop()`**
- Similarly, break your Pi program into functions

What's next?

- Spend time to figure the best control scheme:
  - Move short distance at a time?
  - Move long distance and avoid obstacle automatically?

- For those who did not show me a video, we will do an "on the road" test **today.** Demonstrate the features of your robot!

What's next after what's next?

- Once you passed the "on the road" test, integrate code with group A (deep learning)
  - Discuss the requirement, how the Python program can be merged etc
  - Do it incrementally
- Integration should happen as soon as your vehicle is ready
  - Treasure hunt starts on Friday!

Robot complete…

- All components from here are **optional**
  - i.e. NOT needed for basic model
- Included here for your own learning and maybe useful for advanced model
- You are advised not to include them first if you are new to Arduino ☺
  - If you want to try, keep the code separate and don’t mix with your basic model's code

REAL-TIME SOFTWARE ARCHITECTURES

Real Time Software Architecture

- A “software architecture” = Structure of a software solution
  - Determines whether a piece of software can meet the constraints of a problem, and whether it is actually correct
- Not all real-time systems require an operating system
  - Some alternatives:
    - Round-robin
    - Round robin with interrupts
    - Function queue scheduling
    - Timed loops

Real-Time Software Architectures

ROUND ROBIN

Round Robin: **Basic Idea**

```
while( true )
{
      read sensor 1;
      process sensor 1;

      read sensor 2;
      process sensor 2;

      ......
}
```

Round Robin: **Working Well**

- Round Robin architecture works particularly well when:
  - There are few devices to service
  - There is no long complicated processing associated with the input
  - There are no tight time requirements

Round Robin: **Failing**

- However it fails when:
  - Any device requires attention in less time than it takes for the CPU to go around the loop

- This architecture is also **fragile**:
  - Every single device added to the loop may render the performance unacceptable

Real-Time Software Architectures

ROUND ROBIN WITH INTERRUPTS

Round Robin with Interrupts

- In this architecture:
  - When a sensor has data to send to the CPU, it will ***interrupt*** the CPU
  - The ***interrupt handler*** reads the device and sets a flag
  - The main loop ***polls*** these flags, and takes action if any of the flags is set

Example Code Fragment

```
attachInterrupt( sensor1, ISR1…);
attachInterrupt( sensor2, ISR2…);

while( true )
{
        if (flag1)
                process data1;

        if (flag2)
                process data2;

        ......
}
```

```
void ISR1(...)
{
    data1 = sensor1.read();
    flag1 = true;
}

void ISR2(...)
{
    data2 = sensor2.read();
    flag2 = true;
}
```

Round Robin with Interrupts: **Summary**

- This architecture is an improvement over simple round robin:
  - Devices get attended to immediately ➔ Unlikely to suffer loss of data
- However, it may still take a while for this data to actually be processed!
  - E.g. If data1 takes 400ns to process, then data2 can only be processed after that

Real-Time Software Architectures

FUNCTION QUEUE SCHEDULING

Function Queue Scheduling

- This is similar to Round Robin with Interrupts:
  - Interrupt handlers read the data from the device
  - **BUT** instead of setting a flag, it inserts a pointer to the function to process this data
- The main loop executes functions from the queue in First In First Out order

Example Code Fragment

```
Queue FuncQ;

attachInterrupt( sensor1, ISR1…);
attachInterrupt( sensor2, ISR2…);

while( true )
{
     if ( FuncQ not empty){
          func = FunQ.dequeue();
          (*func)(); //call func
     }
}
```

Can you give one example where this will execute differently then the "Round Robin with Interrupt" architecture?

```
void ISR1(...)
{
     data1 = sensor1.read();
     FuncQ.enqueue( s1Func );
}

//ISR2 is similar

void s1Func( )
{
     //process data1
}

//s2Func is similar
```

Can accommodate task priority by using priority queue instead of FIFO queue.

Real-Time Software Architectures

TIMED LOOPS

Timed Loops

- Similar to round-robin:
  - A while loop repeatedly calls functions to handle processing
  - Each function is called in a fixed order
- **Differences:**
  - Functions are called at **fixed intervals**
  - The main loop enforces the intervals by checking elapsed cycles
- This is useful for routines that must be called at fixed times

Code Fragment: Single Task

```
elapsed = getMilliSec();
while( true )
{
    if ( getMilliSec() - elapsed > 19){
        elapsed = getMilliSec();
        taskA();
    }
}
```

- The ***if-statement*** ensure that the taskA get executed every 20 ms (i.e. 50Hz):
  - **IF** taskA() always finishes under 20ms

- How do we add a **taskB** that only needed once every **100ms**?

Code Fragment 2: Tasks A + B

```
elapsed = 0;
mainLoopCount = 0;

while( true )
{
    if ( getMilliSec() - elapsed > 19){
         elapsed = getMilliSec();
         taskA();

         if (mainLoopCount % 5 == 0)
              taskB();

         mainLoopcount++;
    }
}
```

- How does it work?

Code Fragment 3: Several Slower Tasks

- What if you have several tasks (say X, Y, Z) with a 10Hz frequency?

```
elapsed = 0;
while( true )
{
  if ( getMilliSec() - elapsed > 19){
       elapsed = getMilliSec();
       taskA();

       taskB();
  }
}
```

```
void taskB()
{
    static loopCnt = 0;
    switch( loopCnt ) {
        case 0:
            taskX();
            break;
        case 1:
            taskY();
             ….
        case 4:
             break;
    }
    loopCnt = (loopCnt+1)%5;
}
```

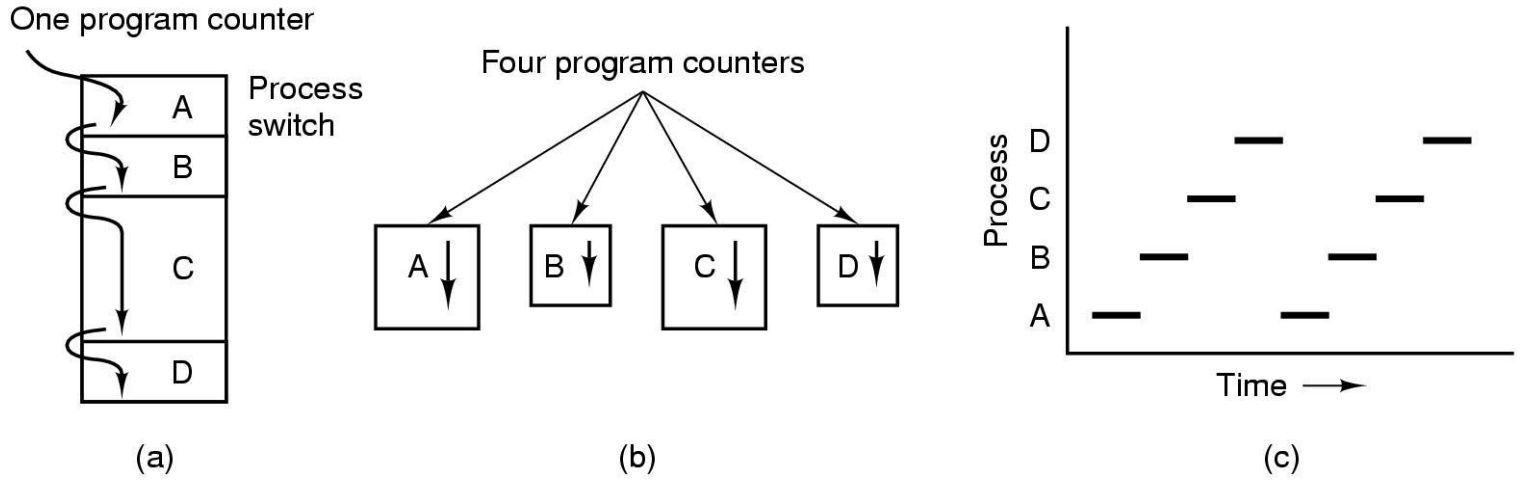
REAL-TIME OPERATING SYSTEMS

Disclaimers

- RTOS is **not absolutely necessary** for our project (we do not have hard deadlines for most of the processing):
  - Hence, we are going to propose it as an **optional feature** to enable the more challenging advanced model
  - Discuss with your group members:
    - You only need to let us know **in the final demonstration (for advanced model)** whether RTOS is used
    - You can decide to use / not to use FreeRTOS at any time:
      - Note that it takes time to recode / learn

The Process (Task) Model

- **Process:**
  - An executing program (includes its running environment)
- On a single-core single processor:
  - At any one time, at most one process can execute.
  - To have multiple processes running, we need to rapidly switch between the processes:

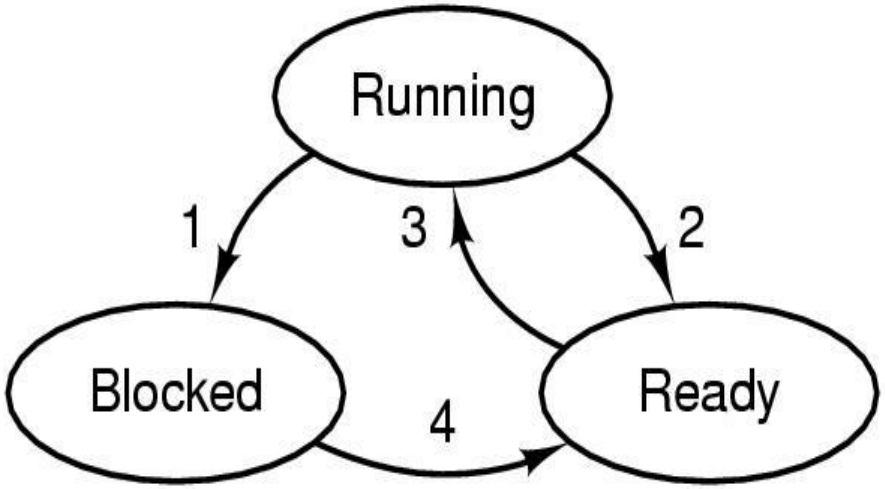


Process States

- A process can be in one of 3 possible states:
  - **Running**
    - The process is actually being executed on the CPU.
  - **Ready**
    - The process is ready to run but not currently running.
    - A “scheduling algorithm” is used to pick the next process for running.
  - **Blocked**
    - The process is waiting for “something” to happen so it is not ready to run yet.
    - E.g. include waiting for inputs from another process.

Process States Transition

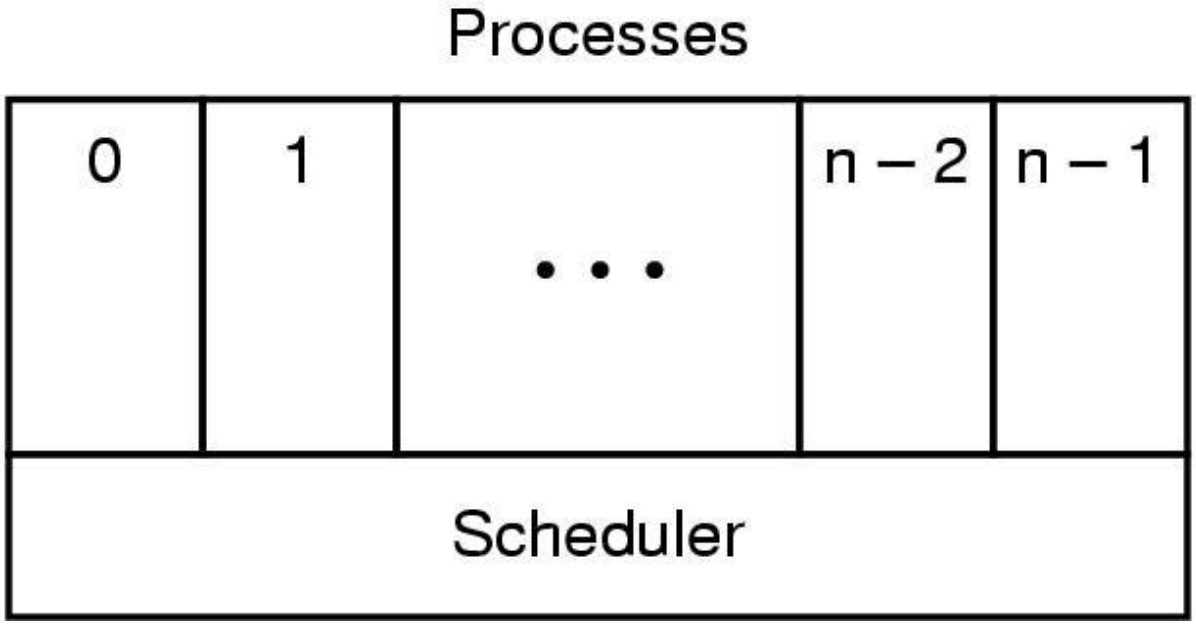
- The diagram below shows the 3 possible states and the transitions between them:



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Process Scheduling

- Scheduler (the lowest layer):
  1. Maintains the list of processes ready to run
  2. Selects (schedules) which process to run next.
     - Subject to “scheduling policies” (later in this lecture)



Real Time Operating Systems

TASK SCHEDULING

Scheduling Policy

- Determines:
  - How to select a task / process to execute next
  - In some cases, determine the task execution duration

- Many possibilities depending on the execution environment:
  - First come first serve, shortest job first, lottery scheduling, multi-level priority queues etc

  - We will focus on a specific subarea:
    - **Real time scheduling**

Real Time Scheduling: **Introduction**

- Tasks have fixed deadlines that must be met
  - Usually ***pre-emptive*** systems

- What if we miss a deadline?
  - ***Hard*** Real Time Systems:
    - System may fails and has severe consequences
  - ***Soft*** Real Time Systems:
    - Mostly just an inconvenience
    - Performance of system is degraded (loss of accuracy, drop in quality, etc)

Task Execution Parameters

- For our example we will assume that each task:
  - Runs for C_i clock cycles each time.
    - This is called the "CPU Time" of the task.
  - Pauses for P_i clock cycles between runs.
    - This is called the "period" of the task.
- We assume 3 tasks P_1, P_2 and $P_3$with the following parameters:

	Ci	Pi
P1	1	4
P2	2	8
P3	3	12

Real Time Scheduling: Definition

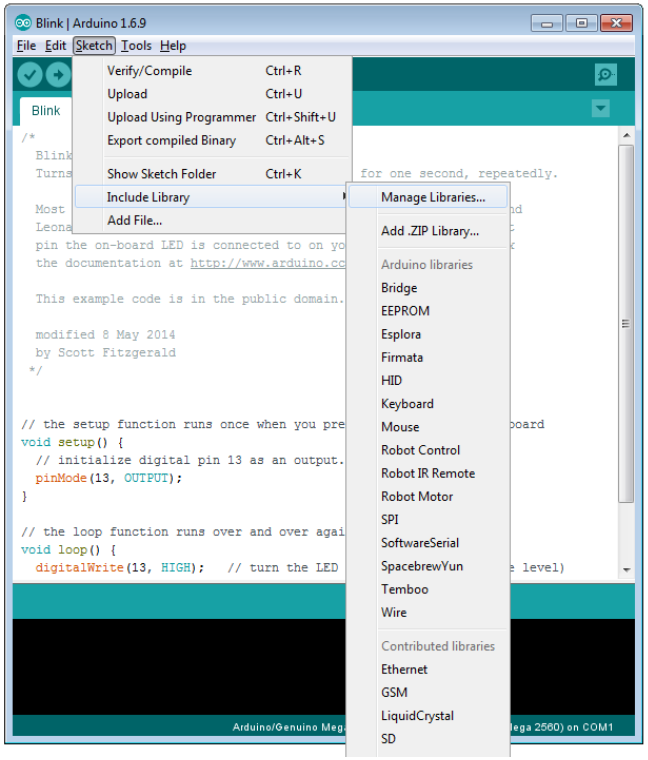
- Real-time Schedulers **must guarantee that tasks complete within time limits**:
  - Time limits are called "**deadlines**"
  - **Assume:**
    - Tasks are periodic with period P_i for task i
    - Tasks use a fixed amount of CPU time C_i each time
  - Deadline D_i is the same as period P_i
    - So, if a task runs at time T_i, it must finish running by $D_i = T_i + P_i$
    - Otherwise, task **i** has missed its deadline

RTOS on Arduino

FreeRTOS – RTOS for Microcontroller

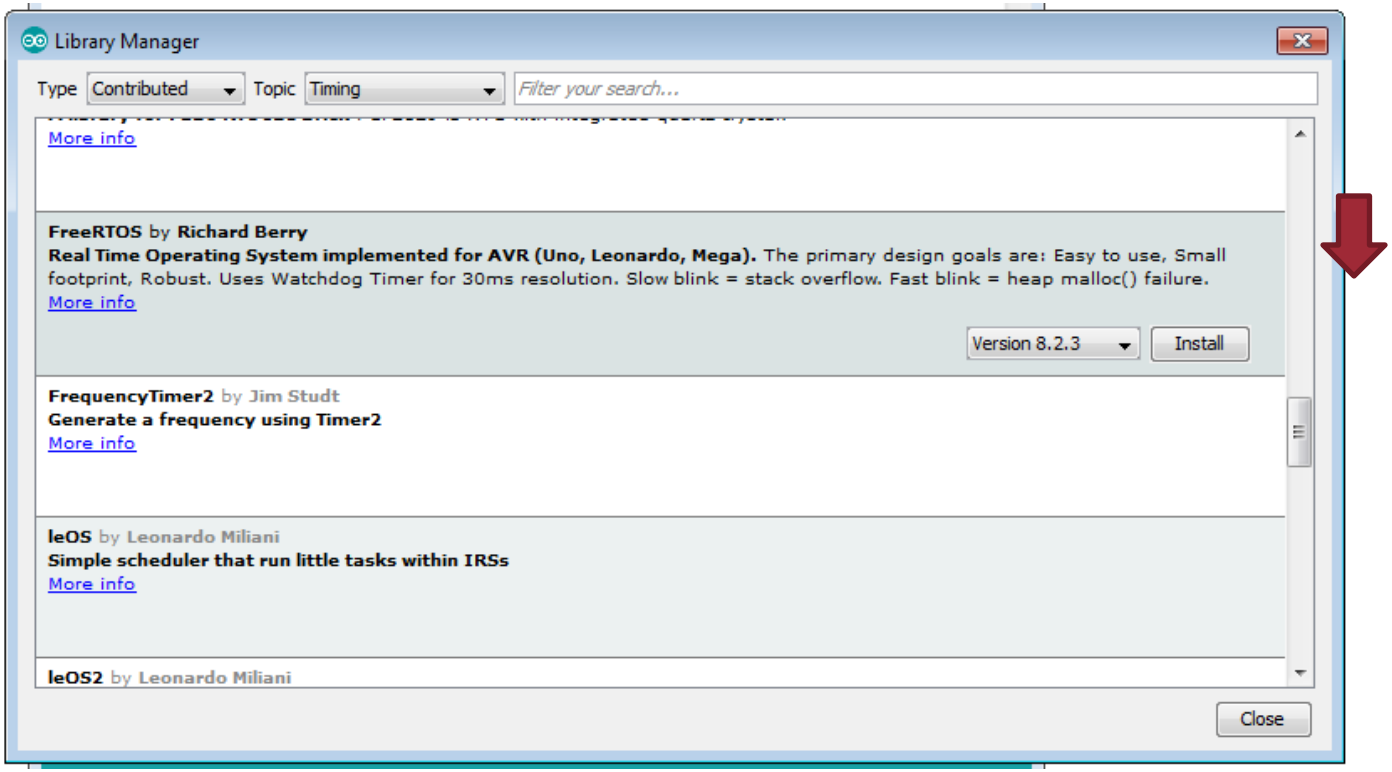
- We are going to demonstrate FreeRTOS for Arduino
  - Popular RTOS library for microcontroller
  - There is a good Arduino version

- Step 1:
  - Click "Sketch→Include library→Manage Libraries"



Installing FreeRTOS Library

2. Select "Type: Contributed", "Topic: Timing"
3. Look for FreeRTOS, select version 8.2.3, and click "Install"



FreeRTOS: Basic Code Structure

- See the sample code "FreeRTOS-Compile.ino"
- With FreeRTOS, we no longer need to use the loop() function for the main processing code
- Instead, we define a number of tasks:
  - Tasks will be executed in multi-tasking fashion (rapid switching between task) by the scheduler
  - See next slide for the code skeleton

FreeRTOS – Arduino: Code Skeleton (1/2)

```
#include <Arduino_FreeRTOS.h>
// Each task is a separate function. An example of 2 tasks:
void TaskA( void *pvParameters );
void TaskB( void *pvParameters );

void setup() {

  // Now set up two tasks to run independently.
  xTaskCreate(
    TaskA   //function that executes the task
    ,   (const portCHAR *)"Task A"   // For human only
    ,   128  // Stack size
    ,   NULL // Parameter passed to the task
    ,   2    // priority
    ,   NULL );  //Task handler for future operation

  xTaskCreate( TaskB,  (const portCHAR *) "TaskB"
    ,   128 ,  NULL,  1,  NULL );

  // The RTOS scheduler is automatically started when
  // setup() finishes
}
```

FreeRTOS – Arduino: Code Skeleton (2/2)

```
void loop()
{
   //Empty. The tasks contains all processing
}

void TaskA(void *pvParameters)  // This is a task.
{
    for (;;){ // A Task shall never return or exit.
          //Do actual work here
    }
}

void TaskB(void *pvParameters)  // Another task
{

  for (;;)  {
          //Do actual work here
  }
}
```

Download "FreeRTOS-Skeleton.ino"

Demo: Give it a try!

- We'll run the code "FreeRTOS-Try.ino"
  - **Guess the output before your run it**

- More demos:
  - A: Change the priority of TaskA to 1 (i.e. same as Task B).
  - B: Undo the change of priority, add delay:
    - vTaskDelay(100 / **portTICK_PERIOD_MS**); //Instead of delay()
    - to both tasks

Reference

- Modern Operating System, 6e
- Colin Tan, CG2271 Lecture Notes
- FreeRTOS reference at:
  - http://www.freertos.org/a00106.html

ROS on Pi

Robotic Operating System

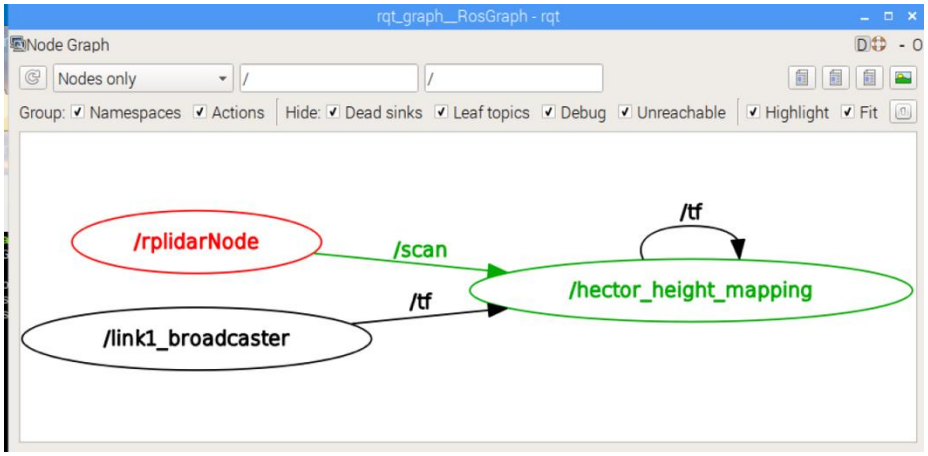
- If you try to explore robotic related development on the web, it is almost inevitable that you will bump into a platform known as ROS sooner or later.
- ROS is a HUGE playground, but also quite daunting place for beginner.
- It is also very painful to use (especially due to dependence on earlier versions of linux).

Robotic Operating System

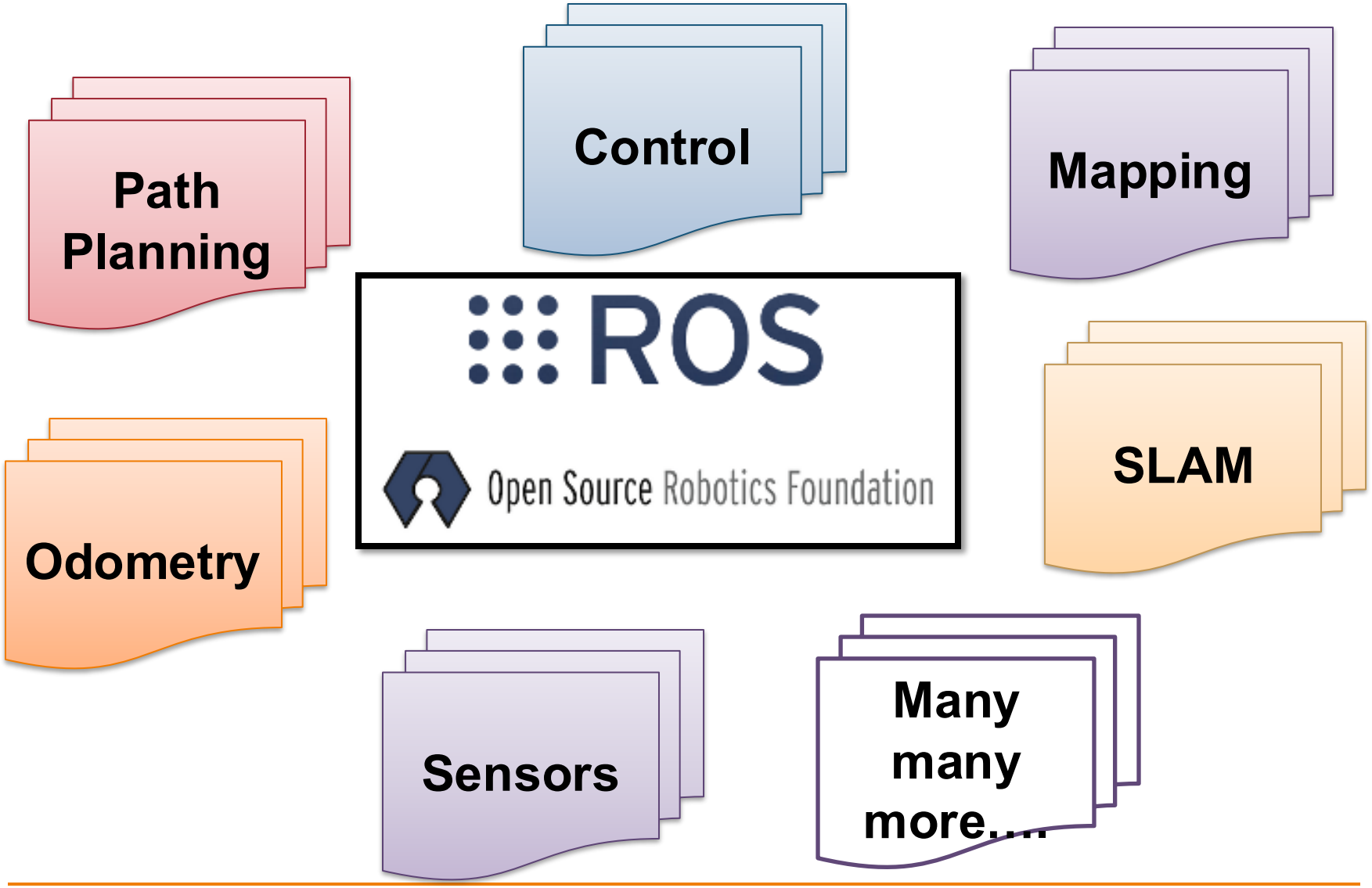
- ROS, despite its name, is not an operating system in the traditional sense.
- Instead, it is more like a middle-ware that enables independent ROS packages to communicate with each other.
- Uses a message broadcast/subscription model.

Robotic Operating System

ROS Terminology	Meaning
Node	An independent ROS package. Can generate message / consume message.
Message	Essentially formatted information. Can be anything generated by a Node.
Topic	Classification of message (i.e. message type).



Thousands of packages



Robotic Operating System

- We don’t recommend using this! It is OVERKILL ☺
- However, we have an image prepared for those who want to play with it in their advanced model.
- There are other middleware alternatives such as ZeroMQ